DAM
Desarrollo de Aplicaciones Multiplataforma
2º Curso

AD
Acceso a Datos

UD 2.3
ANEXO

LOMBOK

IES BALMIS
Dpto Informática
Curso 2026-2027
Versión 1 (09/2026)

1. Clases de datos

Como hemos visto, para definir una clase de datos, debemos incluir al menos:

- Las propiedades que almacenar
- La posibilidad de **Serializar** para conseguir la **persistencia** implementando la clase predefinida **java.io.Serializable** o **java.io.Externalizable**
- Los constructores
  - Constructor sin parámetros
  - Constructor con todos los parámetros
- Los métodos getters y setters
- El método toString sobrecargado parea poder serializar a String

También se ha trabajado la inclusión de este código de forma automática en el IDE NetBeans con el asistente de "**Insert Code**" con la tecla "**Alt+Insert**".

Como ejemplo, se muestra la creación de una clase de datos **Categoria** con las propiedades **codigo** y **descripcion**.

2. Lombok

Las clases de datos con anotaciones de tipo **Lombok** permiten generar código estándar de forma automática o preimplementada.

Project Lombok es una biblioteca para Java que nos ofrece nuevas funcionalidades. Para ello tiene que estar conectado a nuestro compilador, ya que su objetivo es facilitarnos el desarrollo de nuestro código evitándonos tener que escribir ciertos métodos, que van a ser repetitivos y que realmente tampoco aportan lógica al negocio.

El código que puede generar **Lombok** son métodos como **getters/setters, equals, constructores** que nos facilitan el mantenimiento y la limpieza del código.

Las anotaciones más usadas son:

```
@AllArgsConstructor         // => Constructor con todos los argumentos
@NoArgsConstructor          // => Constructor sin argumentos
@RequiredArgsConstructor
        // => Constructor cuando tenemos campos nulos o final
        // => No se incluye si existe @AllArgsConstructor y/o @NoArgsConstructor
@Getter
@Setter
@ToString
@EqualsAndHashCode
@Data   // => @Getter + @Setter + @ToString + @EqualsAndHashCode + @RequiredArgsConstructor
```

Las clases de datos, usen **Lombok** o no, generalmente se **serializan** en algún proceso, es decir, se convierten a algún tipo de flujo de bytes habitualmente como String en diferentes formatos como JSON o CSV. Java recomienda el uso de identificador único denominado **serialVersionUID** para garantizar la consistencia durante el proceso de serialización y deserialización de objetos.

Para serializar correctamente de forma general, y especialmente con Lombok, es añadiremos la propiedad:

```
private static final long serialVersionUID = 1L;
```

Esta propiedad permite comparar versiones de la clase durante la deserialización.
Si el **serialVersionUID** de un objeto serializado no coincide con el de la clase cargada, Java lanzará una expceción **InvalidClassException**.
De esta forma se controla la compatibilidad entre la clase original y la versión actual.

IMPORTANTE - IMPORTANTE

Tanto en **NetBeans** como en otros IDE surgen **problemas sintácticos** no documentados **por no detectar métodos generados por Lombok** y suelen venir por algún aspecto obligatorio en su definición.

Los casos más habituales son:

- No añadir alguna de las **anotaciones** recomendadas en los ejemplos
- No añadir el campo para la serialización
  - **private static final long serialVersionUID = 1L;**

El identificador único de cada clase **serialVersionUID** sirve para:

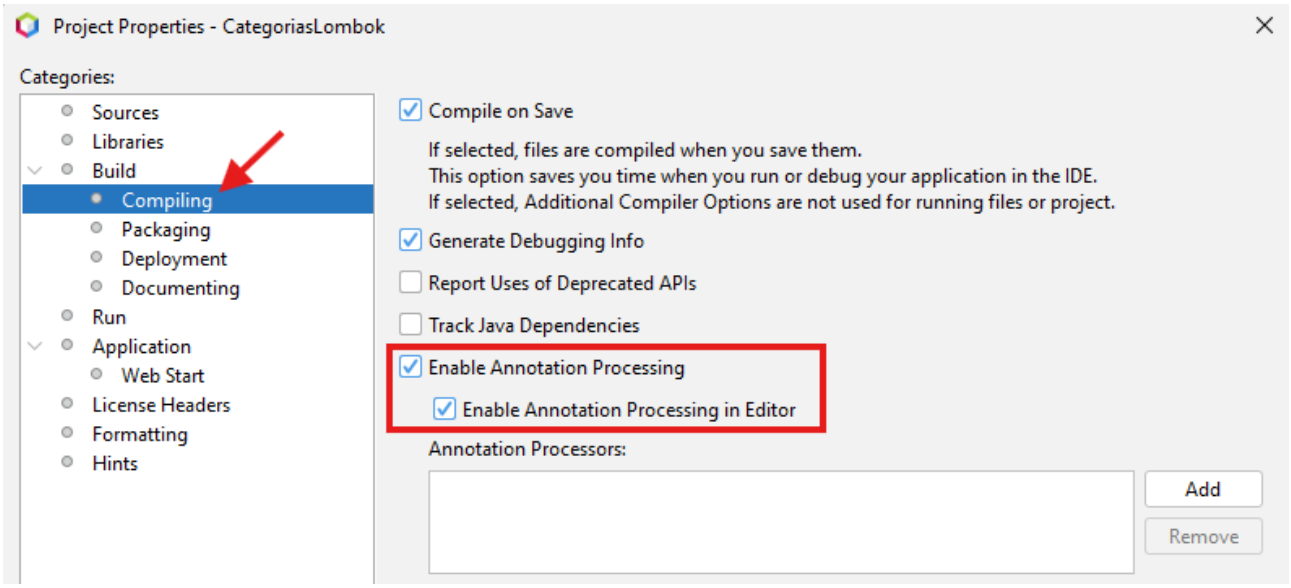
- **Evitar Excepciones de Incompatibilidad:** Si el serialVersionUID del objeto serializado no coincide con el de la clase en el momento de la deserialización, se lanzará una InvalidClassException.
- **Manejo de Versiones:** Facilita la gestión de diferentes versiones de una clase que ha cambiado a lo largo del tiempo.
- **El valor puede ser el mismo en todas las clases**: al ser un identificador para conocer las versiones de una clase de datos podemos utilizar **1L** para todas. Solo tendríamos que cambiarlo en alguna de ellas si en una aplicación en producción ya existen archivos con datos y realizamos cambios que los haga incompatibles.

2.1 Lombok – Java Ant

En proyectos de tipo **Java Ant** añadiremos el **jar** de la librería **lombok**.

Además, se deberá activar en propiedades del proyecto:

- **Enable Annotation Processing**
- **Enable Annotation Processing in Editor**



Categorias

```
Proyecto ejemplo con clase de datos Categoria con los métodos creados manualmente
```

CategoriasLombok

```
Proyecto ejemplo con clase de datos Categoria con los métodos creados utilizando las
anotaciones de Lombok
```

2.2 Lombok – Java Maven

Para que funcionen estas anotaciones en un proyecto de tipo **Java Maven** añadiremos la siguiente dependencia en el **pom.xml**.

pom.xml - lombok

```
<!-- https://mvnrepository.com/artifact/org.projectlombok/lombok -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.48</version>
    <scope>provided</scope>
</dependency>
```

Además, para que se procesen las anotaciones deberemos añadir también al archivo **pom.xml** el **plugin de maven** con la indicación de las anotaciones a procesar.

pom.xml – plugin maven con proceso de anotaciones Lombok

```
<build>
     <plugins>
          <plugin>
               <groupId>org.apache.maven.plugins</groupId>
               <artifactId>maven-compiler-plugin</artifactId>
               <version>3.15.0</version>
               <configuration>
                   <annotationProcessorPaths>
                       <path>
                           <groupId>org.projectlombok</groupId>
                           <artifactId>lombok</artifactId>
                           <version>1.18.48</version>
                       </path>
                   </annotationProcessorPaths>
               </configuration>
          </plugin>
     </plugins>
</build>
```

CategoriasMaven

```
Proyecto ejemplo con clase de datos Categoria con los métodos creados manualmente
```

CategoriasLombokMaven

```
Proyecto ejemplo con clase de datos Categoria con los métodos creados utilizando las
anotaciones de Lombok
```